

Process

to differentiate

$x=1$
 $\text{int } a = \text{fork}();$

if ($a == 0$)
 $x = x + 1;$

$\text{pid} = \text{fork}();$
 $\text{print}(x);$

else

~~wait~~
 $\text{print}(x)$
 parent

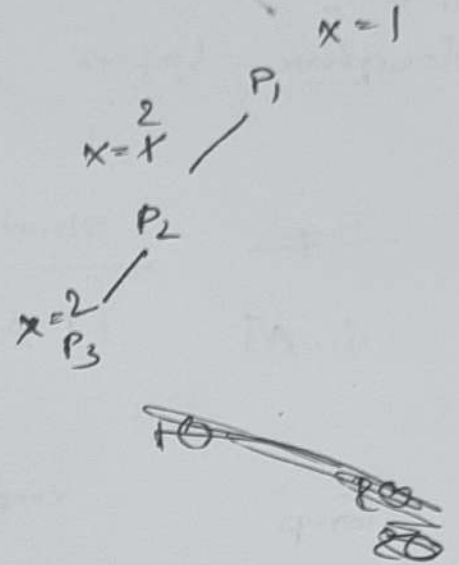
if ($\text{pid} == 0$)

// P_3 $\text{Print}(\text{GrandChild})$

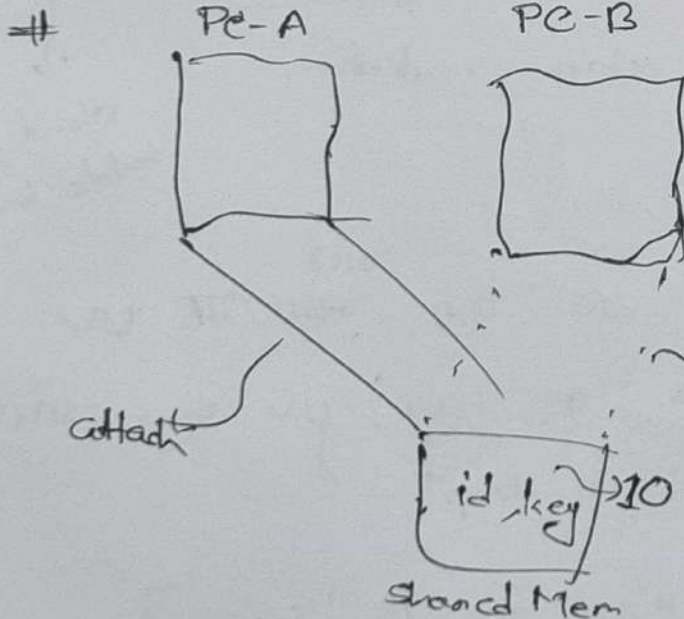
else

$\text{wait}();$
 P_2

$\text{Print}(\text{child})$
~~parent~~



$\Rightarrow$ grand...
 child...
 par...



to read
 need to attach

overlapping

(0) $\text{reg} = \text{age}$
 (1) $\text{reg} = \text{age}$
 (2) $\text{reg} = \text{age}$
 (3) $\text{reg} = \text{age}$
 (4) $\text{reg} = \text{age}$
 (5) $\text{reg} = \text{age}$
 (6) $\text{reg} = \text{age}$
 (7) $\text{reg} = \text{age}$
 (8) $\text{reg} = \text{age}$
 (9) $\text{reg} = \text{age}$
 (10) $\text{reg} = \text{age}$
 (11) $\text{reg} = \text{age}$
 (12) $\text{reg} = \text{age}$
 (13) $\text{reg} = \text{age}$
 (14) $\text{reg} = \text{age}$
 (15) $\text{reg} = \text{age}$
 (16) $\text{reg} = \text{age}$
 (17) $\text{reg} = \text{age}$
 (18) $\text{reg} = \text{age}$
 (19) $\text{reg} = \text{age}$
 (20) $\text{reg} = \text{age}$
 (21) $\text{reg} = \text{age}$
 (22) $\text{reg} = \text{age}$
 (23) $\text{reg} = \text{age}$
 (24) $\text{reg} = \text{age}$
 (25) $\text{reg} = \text{age}$
 (26) $\text{reg} = \text{age}$
 (27) $\text{reg} = \text{age}$
 (28) $\text{reg} = \text{age}$
 (29) $\text{reg} = \text{age}$
 (30) $\text{reg} = \text{age}$
 (31) $\text{reg} = \text{age}$
 (32) $\text{reg} = \text{age}$
 (33) $\text{reg} = \text{age}$
 (34) $\text{reg} = \text{age}$
 (35) $\text{reg} = \text{age}$
 (36) $\text{reg} = \text{age}$
 (37) $\text{reg} = \text{age}$
 (38) $\text{reg} = \text{age}$
 (39) $\text{reg} = \text{age}$
 (40) $\text{reg} = \text{age}$
 (41) $\text{reg} = \text{age}$
 (42) $\text{reg} = \text{age}$
 (43) $\text{reg} = \text{age}$
 (44) $\text{reg} = \text{age}$
 (45) $\text{reg} = \text{age}$
 (46) $\text{reg} = \text{age}$
 (47) $\text{reg} = \text{age}$
 (48) $\text{reg} = \text{age}$
 (49) $\text{reg} = \text{age}$
 (50) $\text{reg} = \text{age}$
 (51) $\text{reg} = \text{age}$
 (52) $\text{reg} = \text{age}$
 (53) $\text{reg} = \text{age}$
 (54) $\text{reg} = \text{age}$
 (55) $\text{reg} = \text{age}$
 (56) $\text{reg} = \text{age}$
 (57) $\text{reg} = \text{age}$
 (58) $\text{reg} = \text{age}$
 (59) $\text{reg} = \text{age}$
 (60) $\text{reg} = \text{age}$
 (61) $\text{reg} = \text{age}$
 (62) $\text{reg} = \text{age}$
 (63) $\text{reg} = \text{age}$
 (64) $\text{reg} = \text{age}$
 (65) $\text{reg} = \text{age}$
 (66) $\text{reg} = \text{age}$
 (67) $\text{reg} = \text{age}$
 (68) $\text{reg} = \text{age}$
 (69) $\text{reg} = \text{age}$
 (70) $\text{reg} = \text{age}$
 (71) $\text{reg} = \text{age}$
 (72) $\text{reg} = \text{age}$
 (73) $\text{reg} = \text{age}$
 (74) $\text{reg} = \text{age}$
 (75) $\text{reg} = \text{age}$
 (76) $\text{reg} = \text{age}$
 (77) $\text{reg} = \text{age}$
 (78) $\text{reg} = \text{age}$
 (79) $\text{reg} = \text{age}$
 (80) $\text{reg} = \text{age}$
 (81) $\text{reg} = \text{age}$
 (82) $\text{reg} = \text{age}$
 (83) $\text{reg} = \text{age}$
 (84) $\text{reg} = \text{age}$
 (85) $\text{reg} = \text{age}$
 (86) $\text{reg} = \text{age}$
 (87) $\text{reg} = \text{age}$
 (88) $\text{reg} = \text{age}$
 (89) $\text{reg} = \text{age}$
 (90) $\text{reg} = \text{age}$
 (91) $\text{reg} = \text{age}$
 (92) $\text{reg} = \text{age}$
 (93) $\text{reg} = \text{age}$
 (94) $\text{reg} = \text{age}$
 (95) $\text{reg} = \text{age}$
 (96) $\text{reg} = \text{age}$
 (97) $\text{reg} = \text{age}$
 (98) $\text{reg} = \text{age}$
 (99) $\text{reg} = \text{age}$

Issue $\rightarrow$ Race Cond

ADALI

Adalimumab INN

(dirty Read type)

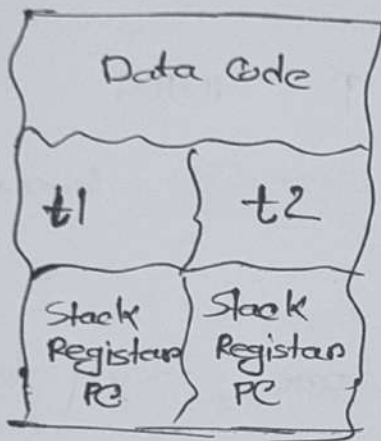
Threads

(Issues of Process)

Memory copy $\rightarrow$ Time consuming

Expensive

For more client not scalable



Stack $\rightarrow$ functions save
stack

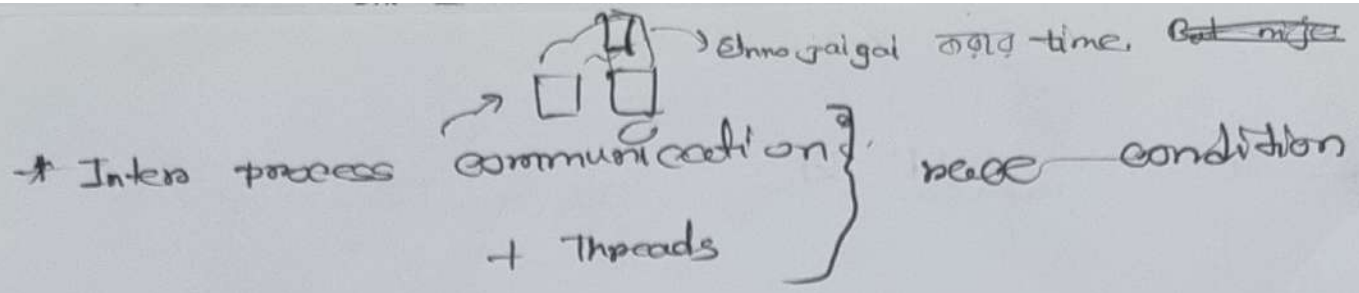
Stack $\rightarrow$ function, local
variable, return

save stack

Same data $\rightarrow$ different Queries $\rightarrow$ create Thread
diff data

Different data $\rightarrow$ different Queries $\rightarrow$ Create Process

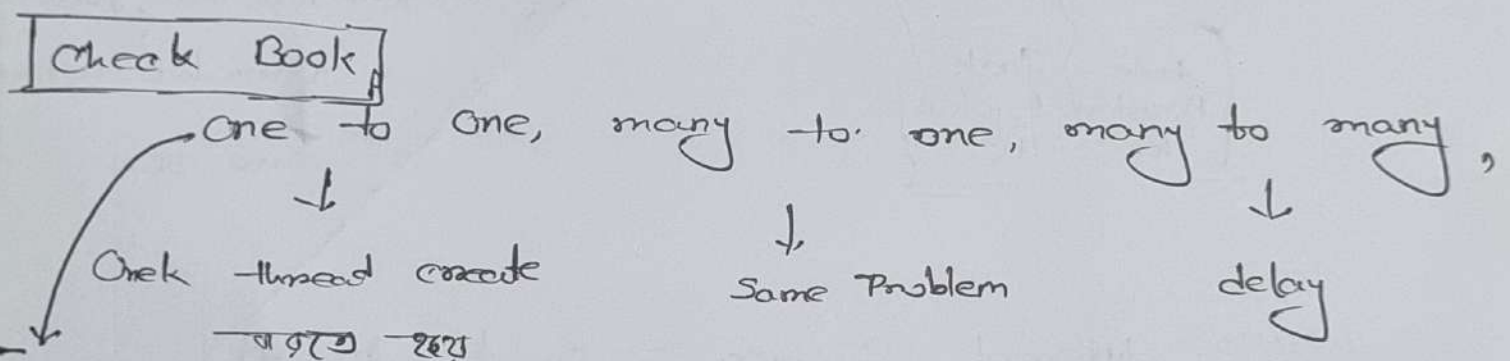
~~By default, process~~



[1, ..., 1M] data, even print

Multithreading Model

Users level thread → Error / Read / Write / Time ↑ / kernel level thread



Used in current level OS → need responsiveness, current machine has overheads

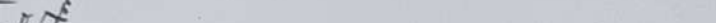
Two level model

↓

M2M, O2O

↓

mixed



Inode bitmap $\Rightarrow [3 \rightarrow 8]$

Overwrite issue solved



after 3 file



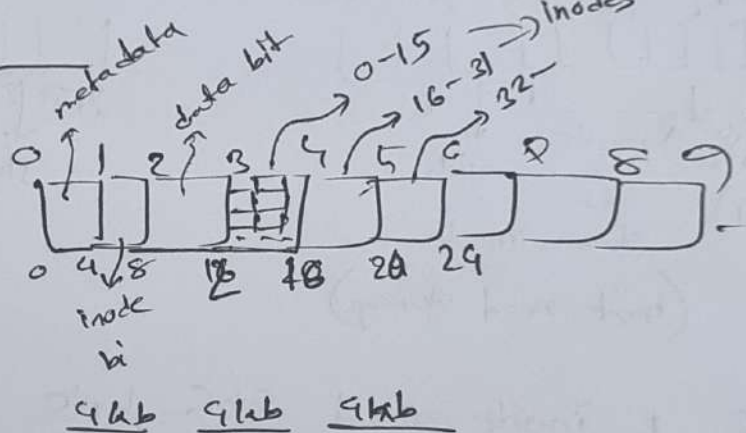
0 full, 0 full, 1 means have space, now store data to 4th part of 3rd block.

data bitmap $\Rightarrow [8 \rightarrow 63]$

each block is actual information stored

32 inode

owners
last accessed time



start $\Rightarrow 3 \times 4 = 12 \text{ kb}$

[0-2] block

32 inode

$$\Rightarrow 32 \times 256 = 8192 \text{ b} \\ = 8 \text{ kb}$$

25 kb

decimal
1

floor
value
for

if, section = 512

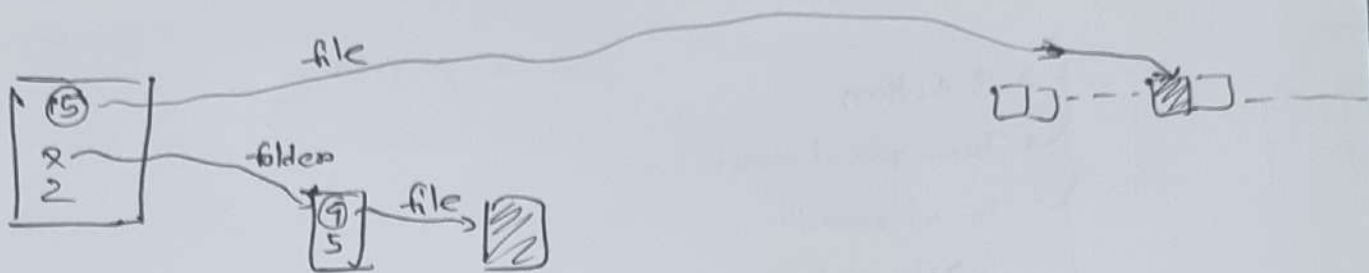
0 - 511 $\rightarrow$ section 0

512 - 1023 $\rightarrow$ section 1

$$\Rightarrow 378 \rightarrow \frac{378}{512} = 0$$

$$\Rightarrow 20 \text{ kb} \rightarrow \frac{20 \times 1024}{512} = 40$$

-folder also need inode (diff inode). Folder has files
folders



⇒ open()

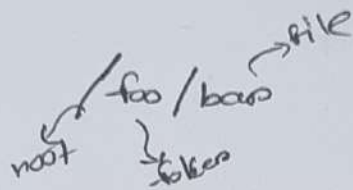
root - inode

↳ root data

⊖ ↳ foo - inode

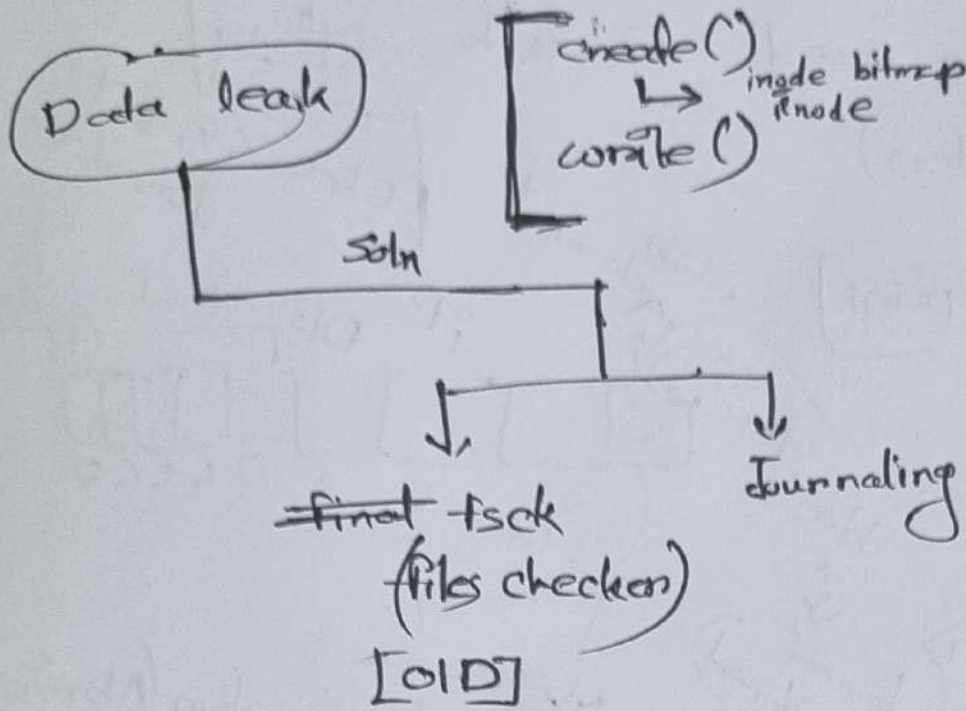
↳ foo data

↳ bar inode



sequence slide

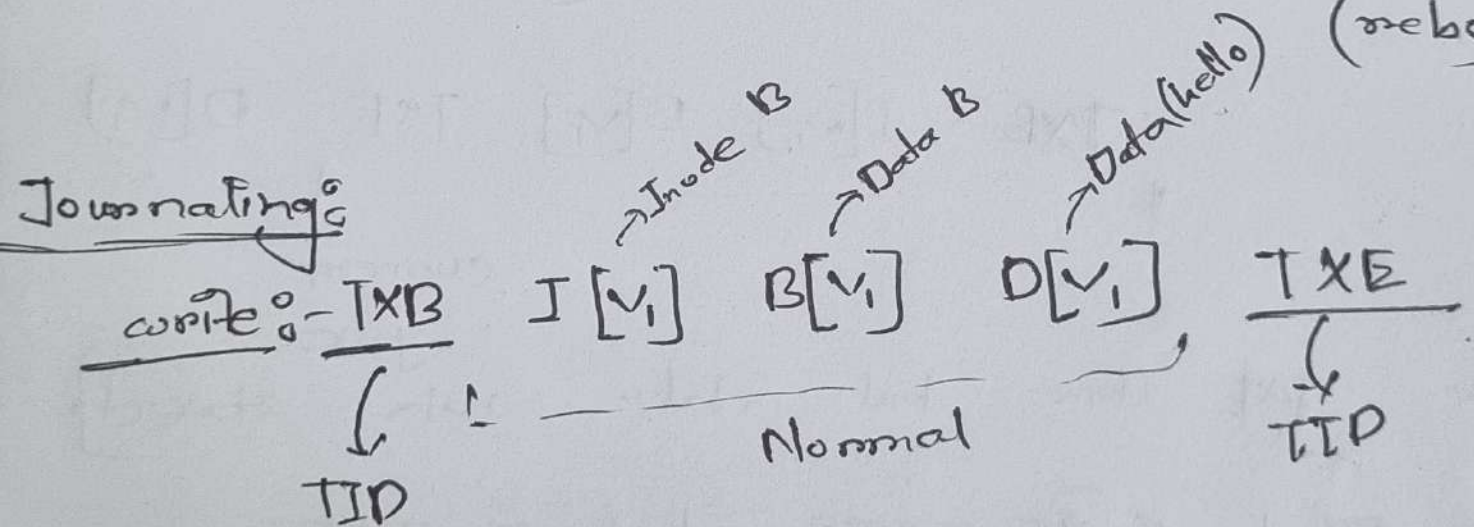
CSE 321



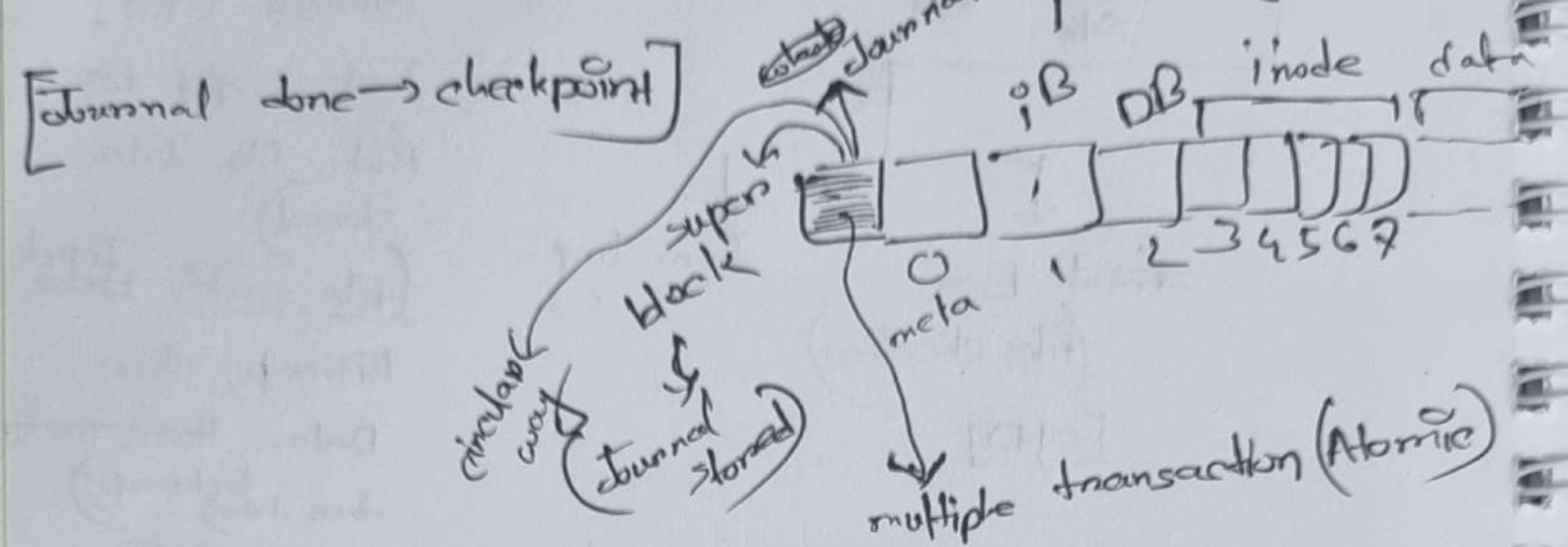
(suppose, data update go time current (start). Inode 3 spot fix But No data stored) (file write: Inode Data) Bitmap, then Data, then inode then data)

fsck: Start at file data check spot data fix go spot go bitmap 0

1. Data bitmap check Cons → too much time needed (reboot)



What if, D done, B done even D done,
but TXE done, no need to do anything
since data written



What if data is ~~deleted~~ current gone?

~~Storing~~ in journal as a transaction
like this (Scheduling) →

TXB I[M] B[M] TXE D[M]

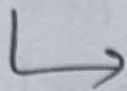
current gone

[Now, TXE done but data not stored]

Soln, First & last TXE always.

[Data large → size issue → Metadata Journaling comes]

Metadat :- (Excluding Data)
Journaling



data
(disk)

TxB

J[m]

B[m]

TxE

data
(disk)

CSE321

(Security)

~~Security Violation~~ → ~~Sahin~~ Changes / Techniques / Tricks

C
I
A
...

Different kind of attack & (Violation)

Masquerading → Pretending to be someone

Replay-attack → bob 100\$, Block the path to the server

Man in the middle →
bob ← Address → Marked
 ↙ Alice ↘

Session hijacking → fib session

FRAT → Remote Access Tool

→ Attacker kno PC to the server, Then attack
 जिसे आसानी से

4 lvl security,

- ① Physical
- ② Application
- ③ OS
- ④ Network

Trojan horse: App A specific program that tends to diff thing then intruded.

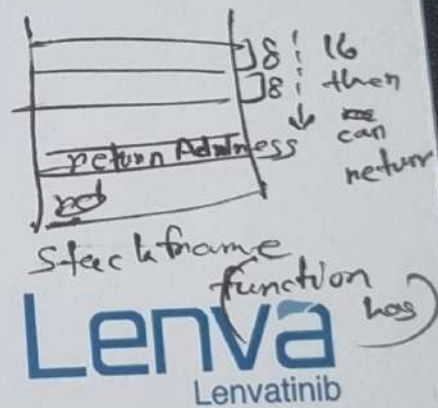
↳ Sneaky Variant: Trojan Mule.

Ransomware: Encrypt your data

trap door: Username, Pass in backend
then taking info, Bd bank dealme
amount money.

Code Injection ✖✖✖

char name[8]
strcpy(name, ".....")
12/18 length
too long



Soranix
Sorafenib

Lenva
Lenvatinib